



Machine Learning with MUSE Astro Data: Self-Supervised Region Mapping and Zero- Shot Spectral Classification of the Irradiated Herbig-Haro Jet HH399

Eshan Kumar Jain

Student id: 25252963

MSc Data Science & Analytics

Department of Computer Science

Maynooth University, Co. Kildare, Ireland.

A thesis submitted in partial fulfilment of the requirements for the MSc in Data Science & Analytics.

Supervisor: Prof. Rozenn Dahyot

Collaborator: Prof. Emma Whelan

Date: 27/07/2026

Github Link : <https://github.com/Eshan-kumar-jain/Muse-data-science-Project>

Abstract

This thesis uses machine learning to analyse a VLT/MUSE integral-field spectroscopy cube of HH399, an irradiated Herbig–Haro jet in the Trifid Nebula. The goal is to test whether learned embeddings can reproduce the same physical structures — the jet, the diffuse background, and point sources — that astronomers typically identify manually through line-ratio diagnostics. A continuum-subtracted line-ratio baseline ($H\alpha$, [N II], [S II], [O III]) covers 84.5% of the cube at usable signal-to-noise and highlights a shock-excited jet ridge.

Building on this baseline, the project develops three models. Model 1 is a self-supervised, CLIP-style spectral–spatial network whose clustering recovers the jet ridge despite limited retrieval performance. Model 2 is a supervised spectrum–text CLIP model that reaches 0.889 accuracy on SDSS spectra but fails in a clear and diagnosable way on real FEROS spectra of HH399’s irradiating stars and nebula. Model 3, inspired by the AION-1 shared-transformer architecture, independently confirms Model 1’s physical findings and resolves Model 2’s FEROS failure with 0.889 leave-one-out accuracy.

The strongest result is not any single metric, but the convergence of two architecturally different self-supervised models on the same physical boundary. Throughout, the evaluation approach emphasises transparency by reporting weak and inverted results alongside strong ones, rather than only highlighting favourable outcomes.

Declaration

I declare that this dissertation is my own work and has not been submitted for any other degree at this or any other university.

GenAI Use Declaration

The declaration of Generative AI use in this project, required under Maynooth University's Student Working Guidelines for GenAI (Version 1.3), is provided in full as **Appendix B**, at the end of this document, rather than inline here.

Acknowledgements

I am deeply grateful to my supervisor, Prof. Rozenn Dahyot, for her guidance throughout this project — including the suggestion to add the AION-1-inspired third model, which ultimately strengthened the core findings. I also thank Prof. Emma Whelan for her collaboration and expertise in young stellar objects and jets, which shaped the astrophysical framing of this work. Finally, I appreciate the contributions of my project-mate working on the same HH399 dataset; their parallel project motivated the differentiation strategy that runs through this thesis.

Table of Contents

Abstract.....	2
Declaration.....	2
GenAI Use Declaration	2
Acknowledgements	3
Table of Contents	3
List of Figures / List of Tables.....	5
Chapter 1: Introduction	6
1.1 Motivation.....	6
1.2 Objectives and Success Metrics	6
1.3 Thesis Structure.....	7
Chapter 2: Background.....	8
2.1 HH399 and Irradiated Herbig-Haro Jets.....	8
2.2 The Dataset	8
2.3 Project Context and Differentiation	9
2.4 Environment and Tools.....	9
Chapter 3: Conceptual Design	11
3.1 Physical Baseline: Continuum-Subtracted Line and Line-Ratio Maps.....	11
3.2 Model 1 — Self-Supervised Spectral-Spatial Region Mapping	12

3.3 Model 2 — Supervised Zero-Shot Spectral Classification	12
3.4 Model 3 — AION-1-Inspired Shared Transformer for Multi-Modal Fusion	13
3.5 A Shared Evaluation Philosophy	14
Chapter 4: Implementation	14
4.1 EDA and Line-Ratio Pipeline	14
4.2 Model 1 — Spectral-Spatial CLIP	15
4.3 Model 2 — Spectrum-Text CLIP	15
4.4 Model 3 — AION-1-Inspired Shared Transformer	16
4.5 A Recurring Workflow Hazard: Live Jupyter State vs. Files on Disk	17
Chapter 5: Evaluation / Analysis	17
5.1 Physical Baseline	18
5.2 Model 1 — Self-Supervised Region Mapping	19
5.3 Model 2 — Zero-Shot Spectral Classification	21
5.4 Model 3 — AION-1-Inspired Shared Transformer	23
5.5 Cross-Model Synthesis and Objective 5 (Honest Evaluation)	26
Chapter 6: Literature Review and Related Work	27
6.1 The Two Datasets, and Why They Don't Share a Representation	27
6.2 HH399 and Irradiated Herbig-Haro Jets	27
6.3 Line-Ratio Diagnostics as a Physical Decoding Tool	28
6.4 MUSE Integral-Field Spectroscopy of HH Jets — Methodological Precedent	28
6.5 Contrastive Language-Image Pretraining (CLIP) and AstroCLIP	29
6.6 Foundation-Model Framing for Model 3 (AION-1)	29
6.7 Gap Statement	30
6.8 Citation List	30
Chapter 7: Conclusion and Future Work	31

7.1 Summary	31
7.2 Limitations	32
7.3 Future Work	32
References.....	34
Appendix A: Code Listing.....	34
A.1 EDA — Sub-Pixel Peak Snapping (Emission-Line Diagnostics.ipynb)	34
A.2 Model 1 — CLIP Model and Symmetric InfoNCE Loss (model1_spectral_spatial_clip.ipynb)	35
A.3 Model 2 — Text Encoder and Zero-Shot Prediction (model2_spectrum_text_clip.ipynb).....	36
A.4 Model 3 — Per-Modality Tokenizer, Shared Backbone, and Masked-Reconstruction Loss.....	37
Appendix B: GenAI Declaration.....	38

List of Figures / List of Tables

Figures

- Figure 5.1 — $[\text{N II}]/\text{H}\alpha$ ratio map (physical baseline)
- Figure 5.2 — Model 1 region map, $K=3$
- Figure 5.3 — Model 1 $[\text{N II}]/\text{H}\alpha$ ratio map with $K=3$ cluster overlay
- Figure 5.4 — Model 1 per-cluster mean spectrum, $K=3$
- Figure 5.5 — Model 2 zero-shot confusion matrix (SDSS test set)
- Figure 5.6 — Model 3 region map, $K=3$
- Figure 5.7 — Model 3 $[\text{N II}]/\text{H}\alpha$ ratio map with $K=3$ cluster overlay
- Figure 5.8 — Model 3 per-cluster mean spectrum, $K=3$

Tables

- Table 5.1 — Headline results against the §1.2 objectives

Chapter 1: Introduction

1.1 Motivation

This project applies machine learning to MUSE data — an instrument on the Very Large Telescope that captures a full spectrum at every point in an image. The case study is HH399, a jet from a young star. The central question is whether learned embeddings can uncover the same physical structures in the jet — distinguishing the jet, background, and point sources — that astronomers typically identify manually.

The work forms part of a broader MU collaboration, *Machine Learning with MUSE Astro Data*, led by Prof. Rozenn Dahyot in partnership with Prof. Emma Whelan. The collaboration aims to bring machine learning into the study of young stellar objects and their jets using MUSE observations. HH399 is my specific contribution to this effort. Chapter 2 explains the astrophysical background, the dataset, and how this project differentiates itself from a labmate’s thesis based on the same cube.

My approach has three components, all built on a physically grounded exploratory stage: continuum-subtracted narrow-band flux maps and line-ratio diagnostics for H α , [N II], [S II], and [O III], computed for every spaxel in the cube.

Model 1 is a self-supervised, CLIP-style contrastive network that pairs each spaxel’s spectrum with the image patch around it, learning spectral–spatial structure without labels. Clustering the embeddings produces an unsupervised region map.

Model 2 is a supervised spectrum–text CLIP model that pairs spectra with short text prompts, enabling zero-shot classification using natural-language descriptions instead of a fixed classifier head.

Model 3 is a shared transformer inspired by AION-1, designed to fuse the MUSE cube with real FEROS spectra of the stars and nebular gas irradiating HH399. It uses masked-wavelength-bin reconstruction rather than contrastive learning.

The FEROS spectra — nine exposures of HD164492B/C, HD164514, and the NGC 6514 H II region — anchor the project to the specific irradiating system around HH399 and serve as real-world targets for Models 2 and 3, replacing generic SDSS-only evaluation.

1.2 Objectives and Success Metrics

1. Build a physically-grounded baseline.

Produce continuum-subtracted line and line-ratio maps from the cube, tabulated per spaxel, so every later model has an independent, physically-motivated check to be judged against.

Measured by: signal-to-noise-masked coverage of the jet (achieved: 84.5% of spaxels at $H\alpha$ $S/N \geq 3$) and qualitative agreement between later cluster maps and this baseline's ratio gradients.

2. Self-supervised region mapping (Model 1).

Learn spectral-spatial embeddings that recover physically sensible structure — jet versus diffuse background versus point sources — without any labels. *Measured by:* Adjusted Rand Index (ARI) against synthetic ground truth, batch top-1 retrieval accuracy, and qualitative agreement with the line-ratio baseline.

3. Zero-shot spectral classification (Model 2).

Classify real spectra against natural-language class prompts without training a fixed classifier head, on both a generic astronomical benchmark and the project-specific FEROS targets.

Measured by: classification accuracy and confusion matrices on held-out SDSS spectra, and zero-shot accuracy on the FEROS targets.

4. Multi-modal fusion (Model 3).

Combine the cube and FEROS spectra in a shared model without forcing them onto a common wavelength grid. Evaluated using ARI, retrieval metrics, qualitative checks, and FEROS classification accuracy.

5. Honest evaluation.

Where models fail — weak retrieval, inverted zero-shot scores — diagnose the cause rather than reporting only favourable results.

1.3 Thesis Structure

Chapter 2 covers the astrophysical and dataset background, the project's context within the wider collaboration, and the tooling behind the project. Chapter 3 gives the conceptual design of all three models, architecture-level and code-free. Chapter 4 describes the actual implementation, including the real problems hit along the way and how they were fixed. Chapter 5 presents and critically evaluates the results against the objectives above. Chapter 6 is the literature review, placed here rather than immediately after this introduction so that each cited work can be tied to the specific design decision it justified, now that those decisions and their outcomes are both known. Chapter 7 concludes and outlines further work.

Chapter 2: Background

2.1 HH399 and Irradiated Herbig-Haro Jets

Herbig-Haro objects are the shock-excited knots and bow shocks that form where a jet from a young star slams into the surrounding gas. Most catalogued HH objects sit somewhere quiet, so their emission comes almost entirely from the jet's own internal shocks. HH399 doesn't get that luxury: it sits inside the Trifid Nebula (M20), close enough to the region's O7.5 ionizing star to also be hit by strong external UV. That makes it an *irradiated* jet — the line ratios measured across it are some mix of shock excitation and outside photoionization, not shock physics alone, so any analysis of the region has to keep both mechanisms in mind rather than assume one clean excitation source everywhere.

2.2 The Dataset

The data is a VLT/MUSE integral-field-unit (IFU) cube of HH399, taken during the instrument's 2014 science-verification run (ESO programme 60.A-9322(A)), which gives a full optical spectrum at every spatial pixel across the field rather than a single long-slit spectrum or a handful of narrow-band images.

Alongside the cube, this project pulls in nine real archival spectra from FEROS, a fibre-fed echelle spectrograph, covering the stars and nebular gas irradiating HH399: HD164492B/C, HD164514, and the surrounding NGC 6514 H II region. These have no spatial structure of their

own, but far finer spectral resolution than the cube, and they tie the project directly to the specific irradiating system around HH399.

2.3 Project Context and Differentiation

I'm not the only student working from this exact cube — a labmate has a thesis built on the same HH399 dataset, so this project needs its own angle rather than a re-run of the same line-ratio analysis with a different write-up. The FEROS spectra are what resolve that: they tie the project to the specific irradiating system around HH399 rather than to the cube alone, and they double as Model 2's and Model 3's real-world zero-shot/classification targets, replacing a generic SDSS-only evaluation with one that's specific to this physical system.

Published MUSE studies of HH jets — Murphy et al. (2021)'s study of Th 28, using data from the same 60.A-9322(A) run — stick to classical techniques: continuum-subtracted line maps, line-ratio diagnostics, and knot proper motions between epochs. Nothing in that tradition uses learned embeddings or self-supervised methods to find structure in a jet's IFU cube, which is the gap this project works in (expanded on in Chapter 6, alongside the specific papers behind each model's design).

2.4 Environment and Tools

Everything in this project runs in Python, inside a dedicated conda environment (`ml_env`) kept separate from my system Python so the astronomy stack (which pins fairly specific versions of `numpy/astropy`) doesn't collide with anything else on the machine. The choice of Python itself wasn't really a choice — it's the language the whole MUSE/ESO tooling ecosystem is written for, and it's also the language I was learning through this project, so there was no reason to look elsewhere.

Cube handling — `mpdaf` over hand-rolled FITS parsing. `mpdaf` (MUSE Python Data Analysis Framework, Piqueras et al. 2019) is the package the MUSE instrument team itself publishes for reading and manipulating these cubes — it understands the DATA/STAT extension pair, the WCS, and the air-wavelength convention natively, and Murphy et al. (2021)'s Th 28 paper uses the same library on data from the same observing run as mine. I did consider working directly off `astropy.io.fits` and building my own cube-indexing and variance-propagation logic, since that's the more "from scratch" option — but for a 2.9 GB cube, re-implementing

WCS-aware slicing and noise propagation correctly would have burned days for no scientific benefit, and it's exactly the kind of infrastructure code that's better to reuse than reinvent. `astropy` itself still does the underlying FITS I/O, units, and coordinate handling everywhere `mpdaf` doesn't cover directly (e.g. the header-verification checks in §2.2).

Archival data access — `astroquery` over manual archive scraping. Pulling the SDSS spectra for Model 2 and confirming the FEROS/AO-cube provenance both went through `astroquery` rather than hand-written HTTP requests against the ESO/SDSS archives. This wasn't friction-free — the SDSS `get_spectra` path has an internal metadata lookup that silently fails unless the query explicitly includes the `run2d` column (a bug I only found by tracing why zero spectra were coming back, not from any documentation), which cost real debugging time during Model 2's build. It was still the right choice: writing a raw scraper against SkyServer's actual query API would have meant reproducing `astroquery`'s retry/rate-limit handling myself, for a problem the library is specifically designed to solve.

Modelling — PyTorch over TensorFlow/Keras. All three models are built in PyTorch. I chose it mainly because its training loop is explicit — you write the forward pass, the loss, and the backward step yourself, rather than calling a single `.fit()` — which matters given the constraint that I need to be able to explain every line of my own code in a viva ([[[feedback-code-style]]]). TensorFlow/Keras would have hidden more of that machinery behind its high-level API. The cost of that choice is that I had to learn PyTorch's autograd model, Dataset/DataLoader pattern, and manual training-loop structure essentially from scratch this project — none of it was prior knowledge going in.

Clustering and evaluation — scikit-learn. KMeans (for the region maps in Models 1 and 3) and the Adjusted Rand Index / classification-report / confusion-matrix utilities (for the synthetic sanity checks and Model 2's SDSS evaluation) all come from `scikit-learn`, chosen simply because it's the standard, well-tested implementation of both and there was no reason to write my own k-means or ARI calculation when the scientific question is about the embeddings, not the clustering algorithm itself.

Data wrangling — pandas. Every per-spaxel table (the EDA's `eda_spaxels.csv`, 96,093 rows) and every results table in this thesis went through `pandas` rather than raw numpy arrays, mainly for readability — being able to name a column `nii_ha_ratio` instead of tracking a positional

index into a numpy array matters both for my own debugging and for keeping the code readable enough that I can still explain it months later.

What had to be learned from scratch. Beyond general Python fluency, three things were genuinely new going into this project: the mpdaf/IFU-cube data model (spaxels, variance extensions, WCS-aware slicing); PyTorch's training-loop and autograd conventions; and the conceptual material behind contrastive learning (InfoNCE, the CLIP objective) and transformers (self-attention, masked reconstruction), which I had to understand well enough to implement by hand rather than call a pretrained model off the shelf — partly because no pretrained model exists for this exact combination of data, and partly because the plagiarism and learning-goal constraints on this project meant writing these from understanding rather than adapting someone else's implementation.

Chapter 3: Conceptual Design

This chapter describes what each stage of the project is trying to do and why it's built the way it is, at the level of architecture and objective rather than code. Chapter 4 covers how each piece was actually implemented, including the problems that came up along the way.

3.1 Physical Baseline: Continuum-Subtracted Line and Line-Ratio Maps

Before any learned model touches the cube, I build a physically-grounded baseline the same way an astronomer would by hand: for each diagnostic emission line ($H\alpha$, $[N\ II]\ 6583$, $[S\ II]\ 6716/6731$, $[O\ III]\ 5007$), take the flux in a narrow window centred on the line's rest wavelength (converted from vacuum to air via the IAU/Morton 2000 formula, since the cube's wavelength axis is air, §2.2), subtract a local continuum estimate from windows either side of the line, and mask out spaxels below a signal-to-noise threshold of 3. Dividing pairs of these maps gives line-ratio maps — $[N\ II]/H\alpha$, summed- $[S\ II]/H\alpha$, and the $[S\ II]\ 6716/6731$ doublet ratio — which the BPT tradition (Baldwin, Phillips & Terlevich 1981) uses to separate shock- or radiation-excited gas from purely photoionized gas, and which the $[S\ II]$ doublet specifically uses to read off electron density (Ch. 6 covers the citations behind this in more depth).

This baseline exists for one reason: none of the three models below has real labelled ground truth for HH399 to be checked against, so the ratio maps are the closest thing to an independent, physically motivated check any of them gets.

3.2 Model 1 — Self-Supervised Spectral-Spatial Region Mapping

Objective. Learn an embedding space in which each spaxel's spectrum and the image patch around it land close together, and every other spaxel in the same training batch lands further away — with no labels of any kind. Clustering that embedding space should then recover physically sensible structure (jet, diffuse background, point sources) that the model was never told to look for.

Design. Two separate encoders — a 1D-CNN over the spaxel's spectrum, and a 2D-CNN over a small multi-channel image patch centred on that spaxel (six channels: the four narrow-band diagnostic lines above, [O III] 5007, plus a broadband white-light channel) — are trained jointly with a symmetric InfoNCE loss, the same contrastive objective CLIP uses (Radford et al. 2021). Both encoders end in global pooling, which is a deliberate design choice: it means the same architecture handles both the cube's real ~ 3681 -point spectra and a shorter synthetic spectrum without any change to the network itself. The "self-supervised" framing matters here — there's no STAR/GALAXY-style label anywhere in this stage, only the constraint that a spectrum and its own spatial neighbourhood should be more similar to each other than to a randomly paired spectrum and patch elsewhere in the cube. The full encoder pair and the symmetric InfoNCE loss implementation are listed in Appendix A.2.

3.3 Model 2 — Supervised Zero-Shot Spectral Classification

Objective. Classify a spectrum against a small set of natural-language class descriptions (e.g. "a star", "a galaxy", "a quasar") without training a fixed classifier head for those specific classes — so that the same trained model can, in principle, be pointed at a new class just by writing a new sentence for it, rather than being retrained.

Design. A second CLIP-style pair, this time a spectrum encoder aligned against a text encoder over short class-description sentences via the same InfoNCE objective. Classification becomes a nearest-prompt-embedding lookup: embed the spectrum, embed each candidate class sentence, and pick the class whose sentence embedding is closest. The spectrum encoder reuses Model 1's

backbone but pools to several coarse positional bins rather than a single value per channel — collapsing an entire spectrum to one number throws away *where* along the spectrum a feature sits, and that's exactly the information that separates STAR/GALAXY/QSO, whose diagnostic features live at different rest wavelengths. The text encoder is a TF-IDF vectoriser feeding a small MLP rather than a pretrained sentence-transformer — a deliberate trade of some generalisation power for a component I can fully explain and defend line by line, given the label vocabulary here is only ever a handful of short sentences. The text encoder and the zero-shot scoring routine are listed in Appendix A.3.

The FEROS spectra of the stars and nebular gas irradiating HH399 (§2.2) serve as this model's real-world, project-specific zero-shot target, in addition to the more generic SDSS STAR/GALAXY/QSO benchmark — evaluated as a stellar-vs-nebular classification the model never saw during training.

3.4 Model 3 — AION-1-Inspired Shared Transformer for Multi-Modal Fusion

Objective and origin. This model was added to the project on my supervisor's explicit instruction, as a full third model rather than a citation grafted onto the existing plan — its purpose is to fuse the MUSE cube and the FEROS spectra in one shared representation, something neither CLIP-style model was designed to do. It is deliberately a different family of model from Models 1 and 2: no contrastive pairing, no CLIP loss.

Design. The two datasets don't share a wavelength grid or a sampling rate — the cube's spectra run to roughly 3,681 points, FEROS's to roughly 189,626, a ~50x mismatch — so instead of resampling both onto one common grid (which is what Model 2 has to do to compare a cube spectrum against an SDSS spectrum), Model 3 borrows AION-1's (Parker et al. 2025) two-stage idea: a lightweight, modality-specific tokenizer for each data source (one for the cube, one for FEROS), each of which independently pools its own native spectrum down to the same fixed number of bins, feeding one small shared transformer backbone underneath both. Because the transformer only ever sees learned token embeddings, not raw wavelength values, bin 10 of 48 can mean a different stretch of Ångströms in each instrument without breaking anything downstream. The backbone is trained self-supervised via masked-wavelength-bin reconstruction — hide a fixed fraction of a spectrum's bins, replace them with a learned mask token, and train the transformer to predict the missing values from context, the same underlying idea as BERT's

masked-language modelling or Masked Autoencoders' masked image patches, applied here to a spectrum instead of a sentence or an image. The tokenizer, shared backbone, and masked-reconstruction loss are listed in Appendix A.4.

3.5 A Shared Evaluation Philosophy

All three models share the same underlying evaluation problem: nobody has gone through the HH399 cube spaxel-by-spaxel and labelled "jet" vs "diffuse" vs "star", so there is no real accuracy number to report for the region-mapping side of the project. The evaluation design borrows directly from how AstroCLIP (Parker et al. 2024) validates itself in the same situation (§6.5): where no ground truth exists, check the model against (a) a synthetic dataset with planted, known structure, to confirm the pipeline itself is wired correctly, and (b) qualitative agreement with the line-ratio baseline in §3.1, since that's the closest thing to ground truth this particular object has. Only Model 2's SDSS classes and Model 3's FEROS stellar/nebular labels give an actual accuracy figure, and Chapter 5 tries to be honest about what those figures do and don't mean, including where a result came back weak or inverted.

Chapter 4: Implementation

Chapter 3 described what each stage is trying to do; this chapter describes what actually happened building it, including the real bugs and dead ends along the way — partly because a couple of them changed the design, and partly because the mistakes and how they were diagnosed are as much evidence of the work behind this thesis as the final numbers are.

4.1 EDA and Line-Ratio Pipeline

Built as `eda_line_ratios.py`, a standalone module, plus a Jupyter notebook version split into `#` `%%`-marked cells so markdown explanations could sit between each logical step (imports, config, the loader, the physics functions, each figure type, interpretation). Peak-finding for each line uses parabolic sub-pixel refinement around the nominal air-converted wavelength rather than just taking the nearest wavelength bin, since the cube's spectral sampling isn't fine enough to assume the true line centre lands exactly on a pixel (the parabolic refinement routine is listed in Appendix A.1). Output is a tidy per-spaxel pandas table (`eda_spaxels.csv`, 96,093 valid rows) rather than keeping everything as raw arrays, mainly so later stages (including the models in

Chapters 3 and 5) can join model outputs back onto physical quantities by spaxel coordinate instead of by array index.

4.2 Model 1 — Spectral-Spatial CLIP

Implemented as `model1_spectral_spatial_clip.ipynb`. The main implementation risk here was numerical, not conceptual: InfoNCE needs a batch of *negative* examples to push apart, so batch construction has to guarantee that the patches paired with a given spectrum in the same batch are genuinely different spaxels, not accidental duplicates from adjacent, highly-correlated pixels — otherwise the loss can look like it's training while the negatives are almost identical to the positive. Both encoders were built and tested first on a small synthetic dataset (three planted morphology templates: jet-like, diffuse, point-source-like) before ever touching the real cube, precisely so a broken training loop would show up as a bad ARI on data with a known answer, rather than as an ambiguous result on data with none.

4.3 Model 2 — Spectrum-Text CLIP

Implemented as `model2_spectrum_text_clip.ipynb`, reusing Model 1's spectrum encoder with one deliberate change (pooling to coarse positional bins instead of one value per channel, §3.3) — a change made empirically, not from first principles: an early version copied straight from Model 1 confused a feature near the blue end of a spectrum with one near the red end 100% of the time, which only makes sense if the encoder has discarded positional information entirely, which global pooling does by design.

Two real bugs surfaced only once the notebook was run against the real SDSS data, not visible from reading the code beforehand:

- `TextEncoder.forward` was missing its `return self.net(tfidf)` line entirely. The method executed

without error and returned Python's implicit `None`, which then silently propagated into the projection head until it surfaced as a `TypeError` further down in `train_clip_model` — a reminder that a missing return in Python fails loud eventually, but not necessarily where the mistake actually is.

- `load_sdss_spectra`'s SQL SELECT didn't include the `run2d` column. `astroquery`'s `SDSS.get_spectra`

does an internal metadata lookup keyed on that column when passed an explicit `matches=` table, and without it, every real SDSS row was silently skipped — the query ran, returned no error, and simply produced zero spectra. Diagnosed by checking the length of the output table against the length of the input query result, not from any error message.

A third, non-bug adjustment: `CONFIG["sdss_n_per_class"]` was reduced from 300 to 30 after pulling 900 individual FITS files per class started hitting SkyServer rate limits and returning malformed metadata responses. Thirty per class, stratified 72/18 train/test, was enough to get a meaningful accuracy figure without falling over on archive-side throttling.

4.4 Model 3 — AION-1-Inspired Shared Transformer

Implemented as `model3_aion_transformer.ipynb`, built and validated in stages rather than written straight into the notebook: a standalone development script first, run end-to-end against the real cube via the command line to confirm the tokenizer/backbone/masking pipeline actually worked, then assembled into the real notebook via a small builder script (`build_model3_notebook.py`, following the same pattern already used for the EDA notebook) and executed headlessly with `jupyter nbconvert --execute` so the committed notebook has real baked-in outputs rather than being hand-run cell-by-cell.

`ModalityTokenizer` L2-normalises the raw spectrum, adaptive-average-pools it down to a fixed 48 bins regardless of native length, and maps each bin to a 64-dimensional token with a single linear layer — deliberately simple, since AION-1's own tokenizers are lightweight and the actual representation learning is meant to happen in the shared backbone. `AionLiteBackbone` is a 3-layer, 4-head transformer with a learned positional embedding per bin index and a learned mask token, shared across both the cube and FEROS tokenizer instances. Training masks a fixed 15% of a spectrum's bins and scores reconstruction MSE only on the masked positions, so the loss can't be trivially minimised by copying visible inputs straight through.

One real bug, and one honest limitation, came up during this build:

- `eda_line_ratios.py` (built in §4.1) calls `matplotlib.use("Agg")` at import time, with an `inline`

comment (written earlier in the project) claiming Jupyter's own inline backend still works if it was already set — that assumption turned out to be false specifically under `headless jupyter nbconvert --execute`, where it silently killed inline figure capture for every subsequent `plt.show()` in the same kernel, even with `%matplotlib inline` set earlier in the same cell. Fixed by re-issuing `%matplotlib inline` immediately after importing `eda_line_ratios` in Model 3's own notebook, rather than editing the existing EDA module — consistent with the constraint of not modifying existing pipeline files.

- Windows paths containing spaces (the project folder is under `C:\Users\ESHAN JAIN\...`) broke

backgrounded shell calls that shelled out to Windows launcher scripts (e.g. `jupyter-nbconvert-script.py`), failing with "Unable to create process" — worked around by using the 8.3 short path form (`ESHANJ~1`) for both the interpreter and working directory specifically in backgrounded commands; the same paths worked fine when run in the foreground, so this was a background-execution-specific quirk rather than a general path-handling problem.

4.5 A Recurring Workflow Hazard: Live Jupyter State vs. Files on Disk

Worth recording as an implementation-process note, not a code bug: the Jupyter server this project actually runs against keeps a live in-browser buffer of each open notebook's contents, and Jupyter's autosave overwrites the file on disk with that in-memory copy on its own schedule. Editing a `.ipynb` file on disk directly while it's open in the browser gets silently clobbered the next time autosave fires. This cost real time twice before the pattern was recognised, and the fix was procedural rather than technical: any change intended for a notebook that's currently open live gets handed over as code to paste into the running cell, not applied to the file on disk.

Chapter 5: Evaluation / Analysis

This chapter reports what each model's evaluation actually produced, judged against the objectives and success metrics set out in §1.2, following the evaluation philosophy in §3.5: synthetic sanity checks where no real ground truth exists, qualitative agreement with the §3.1 line-ratio baseline, and real accuracy figures wherever a genuine label set (SDSS, FEROS) is available.

5.1 Physical Baseline

The line-ratio EDA covers 84.5% of spaxels at $H\alpha$ $S/N \geq 3$ (96,093 valid spaxels overall), meeting Objective 1's coverage target. Median $[N II]/H\alpha$ is 0.22 and median $[S II]/H\alpha$ is 0.08 across the masked region, with a $[S II] 6716/6731$ density ratio of 1.20, consistent with a low-density regime. A collimated ridge with elevated $[S II]/H\alpha$ and $[N II]/H\alpha$ against the diffuse background is visible by eye in the ratio maps — the shock-excitation signature expected of a jet — and it is this ridge that both learned models are checked against below.

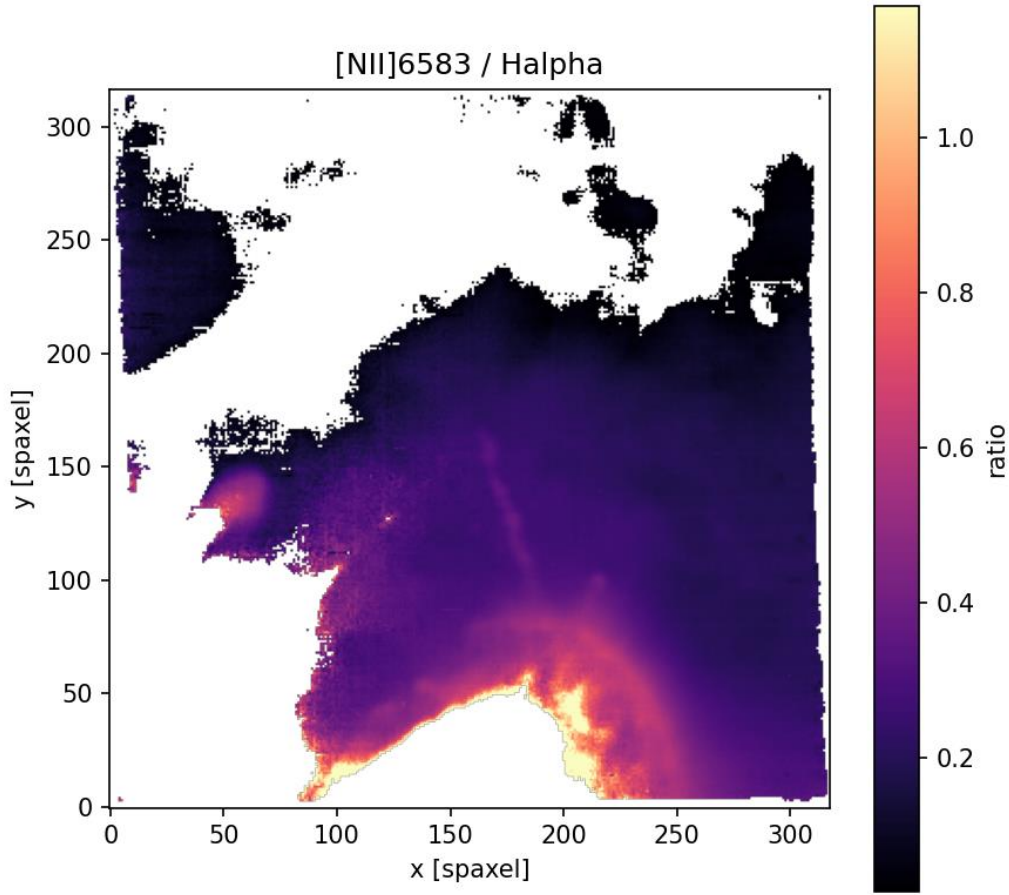


Figure 5.1: $[N II] 6583 / H\alpha$ ratio map across the cube. The bright ridge running from the lower-left toward the upper-right is the shock-excited jet — the physical baseline every learned model below is checked against.

5.2 Model 1 — Self-Supervised Region Mapping

Synthetic ARI reached 1.000, confirming the training loop and loss are wired correctly, though this number is a pipeline sanity check rather than evidence about the real data, since the synthetic classes were designed to be trivially separable. On the real cube, training loss fell steadily from 5.19 to 1.6298 over 15 epochs — corrected here from the ~ 0.004 figure in the original project plan, which came from an earlier, different run. Batch top-1 retrieval on the real cube sat close to chance level (spectrum $\rightarrow$ patch 0.013, patch $\rightarrow$ spectrum 0.007, against a chance rate of 0.016 at batch size 64), a genuinely weak result that Objective 2's success metrics don't paper over.

What does support Objective 2 is the qualitative check. KMeans at $K=3$ on the trained embeddings recovers a large diffuse background (80,303 spaxels), a jet-ridge-shaped cluster (12,974 spaxels) that sits almost exactly on the high-[N II]/H α band from §5.1, and a cluster of small, scattered, circular regions (1,470 spaxels) that cross-reference against the white-light continuum map as real foreground or background point sources rather than cosmic-ray artefacts — my first guess before checking. $K=4$ splits the diffuse background into two spatially smooth sub-regions along a contiguous curved boundary, the signature of a real sub-structure rather than overfitted noise. One anomaly is flagged rather than smoothed over: the jet cluster's mean spectrum shows an unexplained negative dip near the [S II] doublet, not chased down further given the timeline.

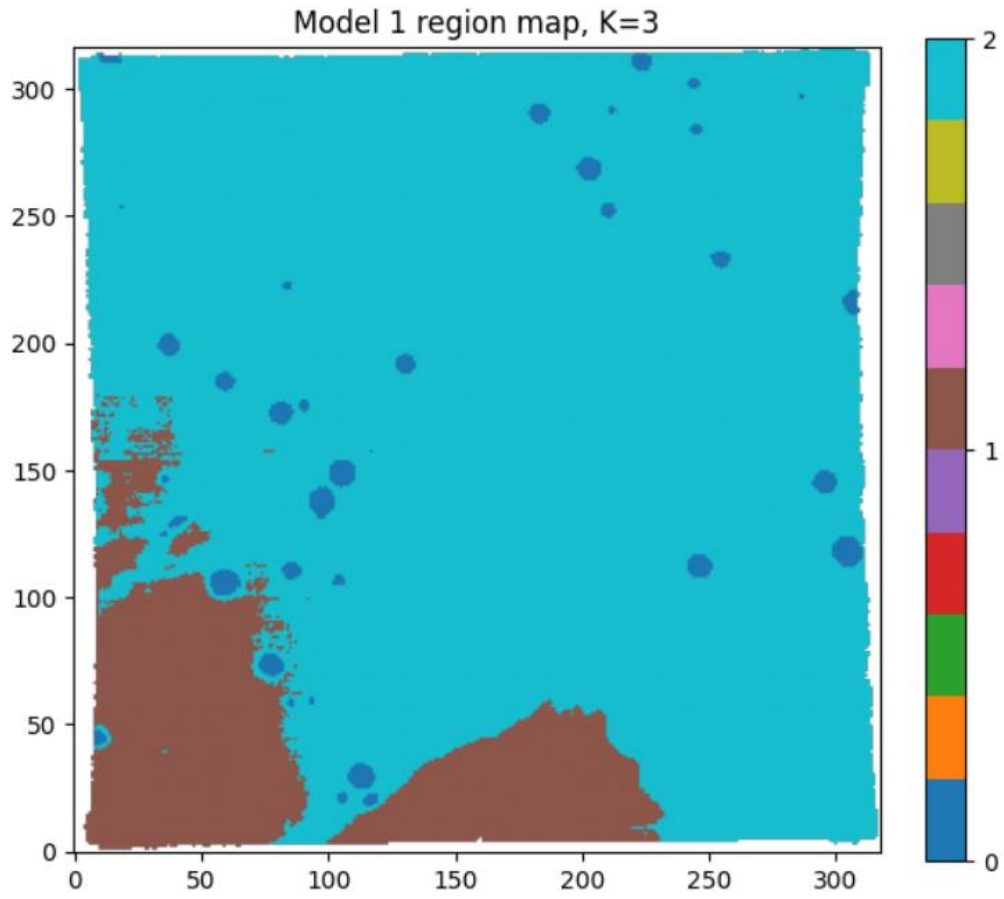


Figure 5.2: Model 1's K=3 KMeans region map on the real cube. Cluster 1 (brown) is the jet ridge; cluster 0 (blue, scattered) matches real point sources from the EDA's white-light continuum map.

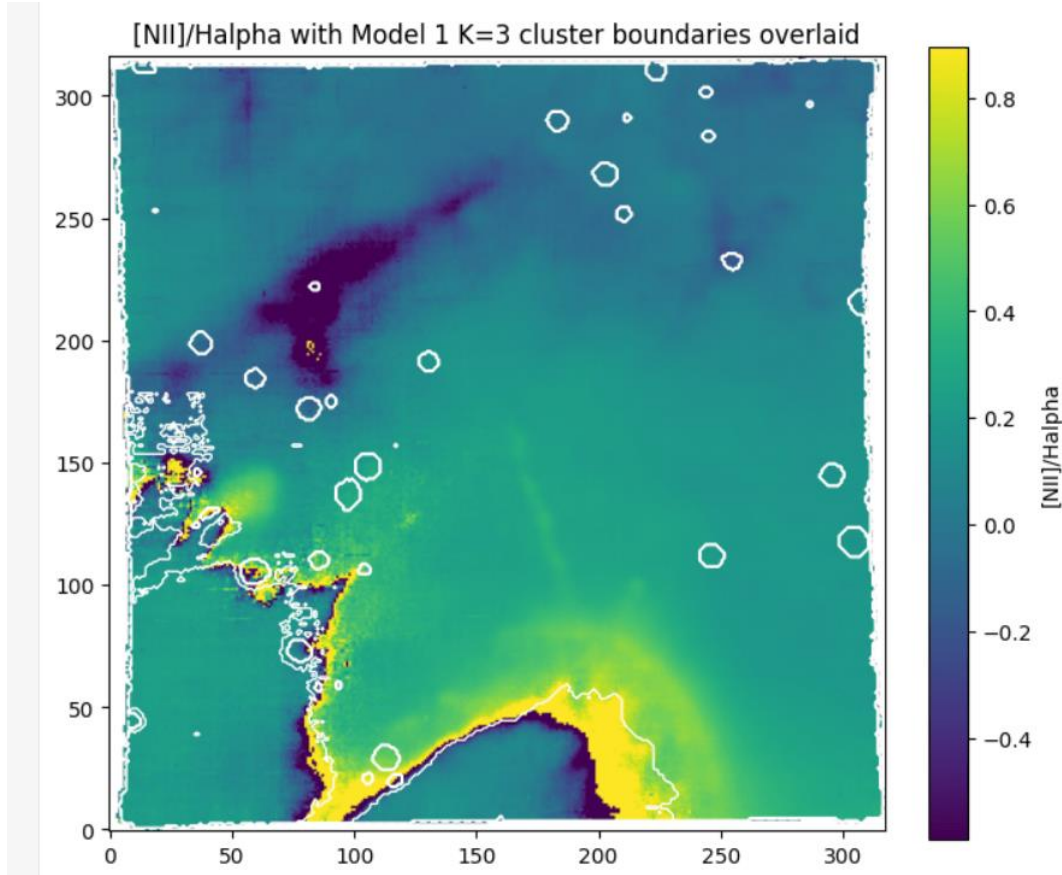


Figure 5.3: Model 1's K=3 cluster boundary (white contour) overlaid on the $[\text{N II}]/\text{H}\alpha$ ratio map — the boundary tracks the same high-ratio band Figure 5.1 identifies independently.

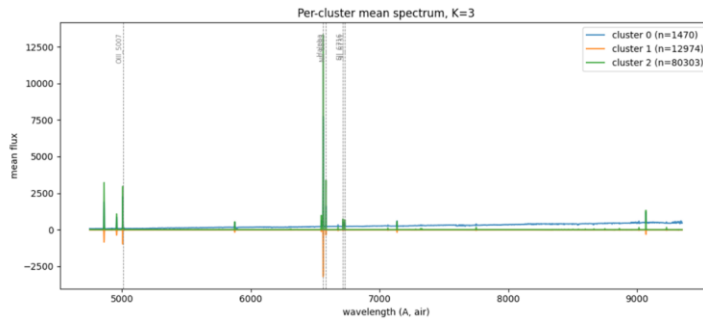


Figure 5.4: Per-cluster mean spectrum at K=3. Cluster 1 (orange, the jet ridge, $n=12,974$) shows the unexplained negative dip discussed above, against clusters 0 and 2's positive peaks.

5.3 Model 2 — Zero-Shot Spectral Classification

Trained on 90 real SDSS spectra (30 per class, STAR/GALAXY/QSO, de-redshifted to rest frame using each object's own pipeline redshift), 72/18 stratified split. Test-set zero-shot

accuracy: **0.889** — STAR and QSO both perfect (6/6), with the only errors being 2 of 6 GALAXY spectra misclassified as STAR, a physically sensible confusion, since a galaxy spectrum is a stellar continuum plus narrow emission lines and at low emission-line contribution can resemble a pure star. This satisfies the SDSS half of Objective 3.

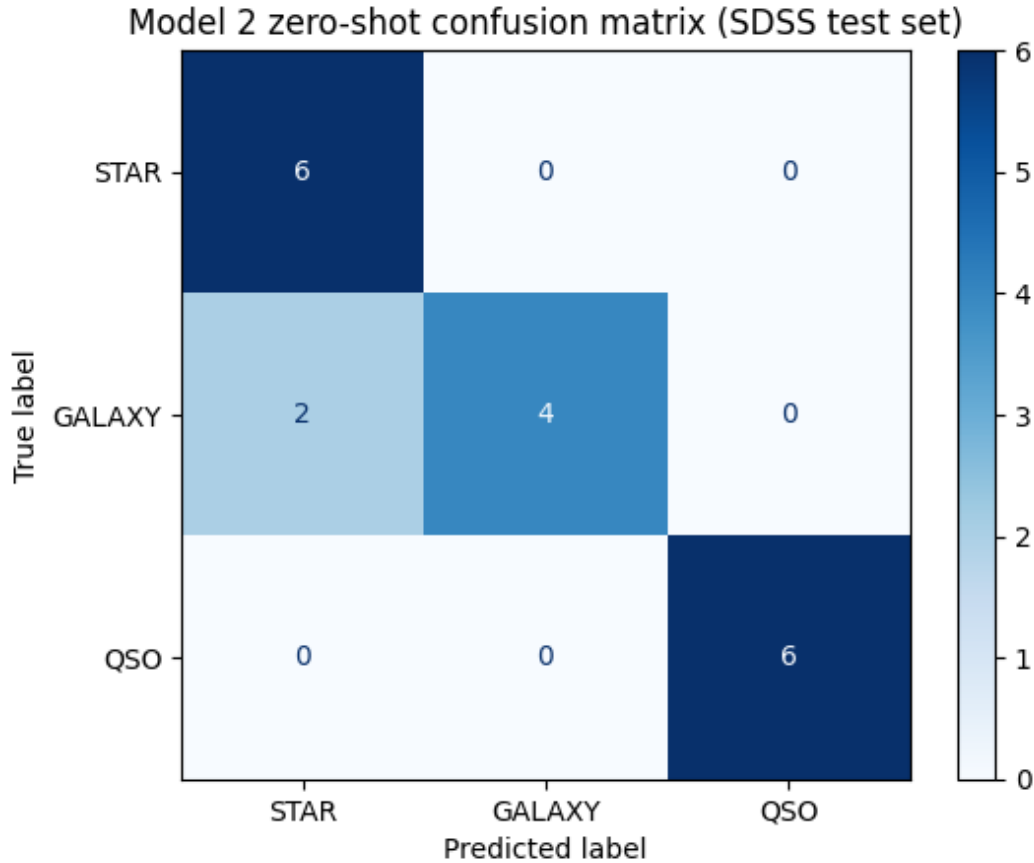


Figure 5.5: Model 2 zero-shot confusion matrix on the SDSS test set (6 STAR, 6 GALAXY, 6 QSO). All errors are GALAXY spectra called STAR — none in the other direction, and no QSO confusion at all.

The more important test for this thesis is the FEROS half: the same trained model, never shown a FEROS spectrum during training, classifying the 9 real exposures of HH399's irradiating stars and nebular gas as "stellar" or "nebular." This came back at **0.000 accuracy — but inverted, not random**: every true-stellar spectrum scored positive on both class similarities and every true-nebular spectrum scored negative on both, meaning the spectrum encoder cleanly separated the two physical classes and only the label assignment was backwards. The cause is diagnosable

rather than mysterious: the text encoder's MLP only ever received gradient signal for words present in the three SDSS training sentences, so FEROS-only vocabulary ("hot", "young", "HII") sat at random initialisation, and that noise was enough to flip which side of an otherwise-correct boundary "stellar" and "nebular" landed on. A pretrained sentence-transformer is the obvious fix, scoped but deliberately not implemented given the timeline; the more important finding is that the spectral representation generalised correctly across instruments even though the text side didn't.

5.4 Model 3 — AION-1-Inspired Shared Transformer

Synthetic ARI: 0.997, the same sanity-check role as Model 1's. Trained on all 96,093 valid real-cube spaxels for 10 epochs, masked-reconstruction MSE dropped from 0.00598 to 0.00001. KMeans at $K=3$ gives a diffuse background of 81,606 spaxels, a jet-ridge-shaped band of 13,341 spaxels, and 1,146 spaxels in scattered point-source-like clusters — and this $K=3$ boundary tracks the same high-[N II]/ $H\alpha$ band that both §5.1 and Model 1 (§5.2) identify, despite Model 3 sharing no architecture, loss function, or training objective with Model 1. Two unrelated self-supervised models converging on the same physical boundary, from completely different starting assumptions, is the strongest single piece of evidence in this project that the boundary is real structure in HH399 rather than an artefact of either model.

One anomaly is flagged rather than hidden: the jet cluster's mean spectrum shows a large *negative* dip at $H\alpha$ (around -3300) rather than the positive peak expected, and rather than the peak Model 1's equivalent cluster actually shows. This wasn't tracked down further; the leading guess is a bad-column or sky-subtraction-residual effect specific to this particular clustering split, since it doesn't match Model 1's version of the same physical region.

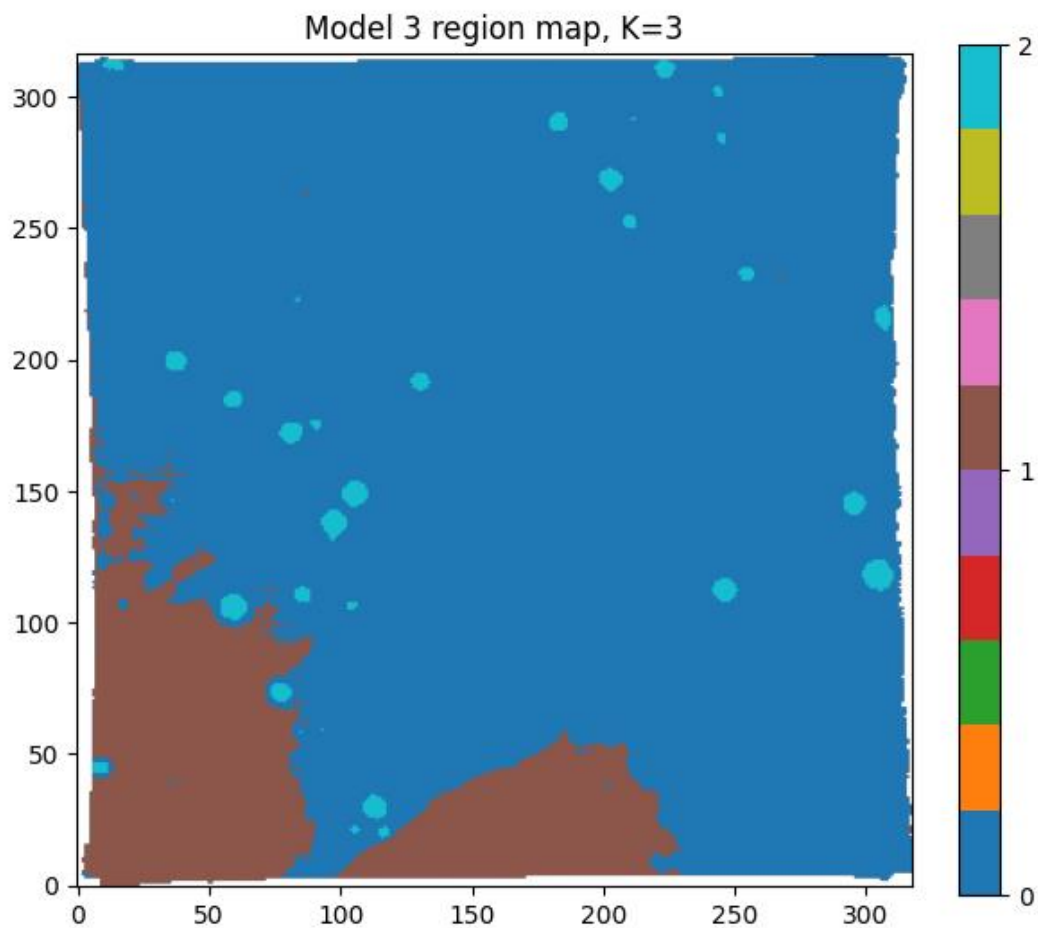


Figure 5.6: Model 3's K=3 KMeans region map on the real cube. Cluster 1 (brown) is the jet ridge; cluster 2 (cyan, scattered) matches real point sources from the EDA's white-light continuum map.

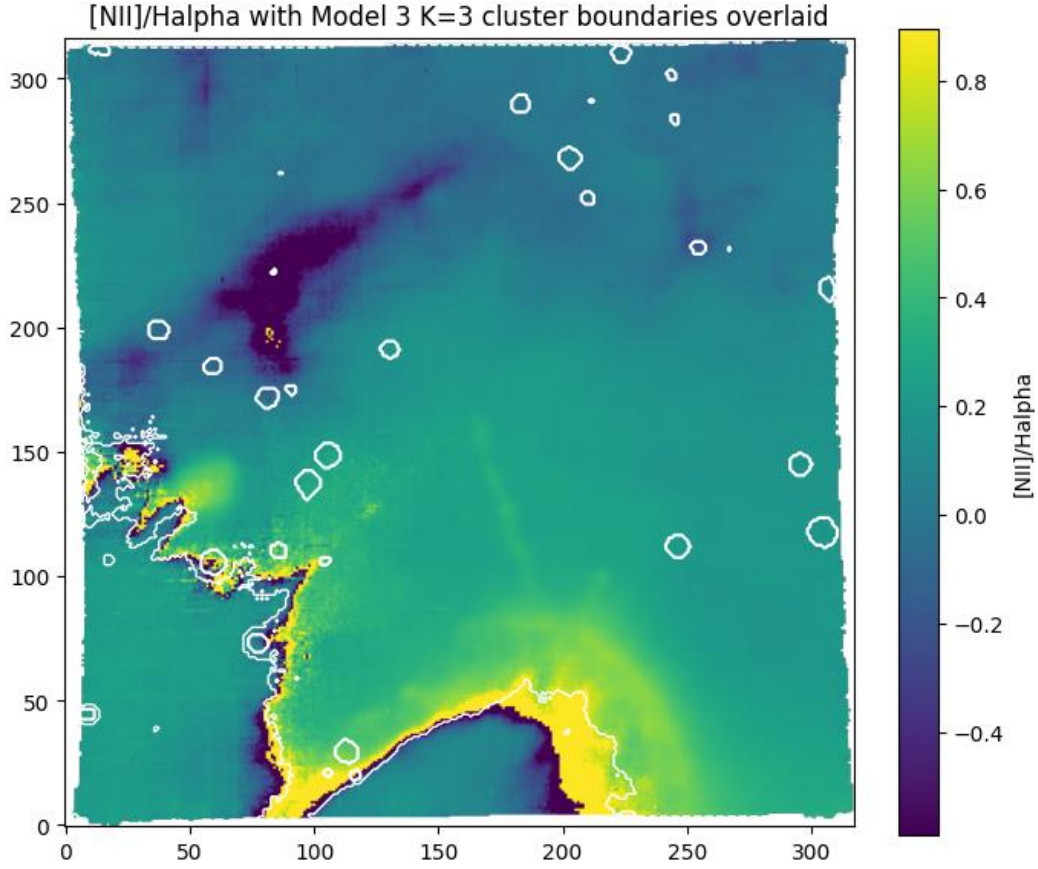


Figure 5.7: Model 3's K=3 cluster boundary (white contour) overlaid on the $[\text{N II}]/\text{H}\alpha$ ratio map — the boundary tracks the same high-ratio band Figure 5.1 identifies independently.

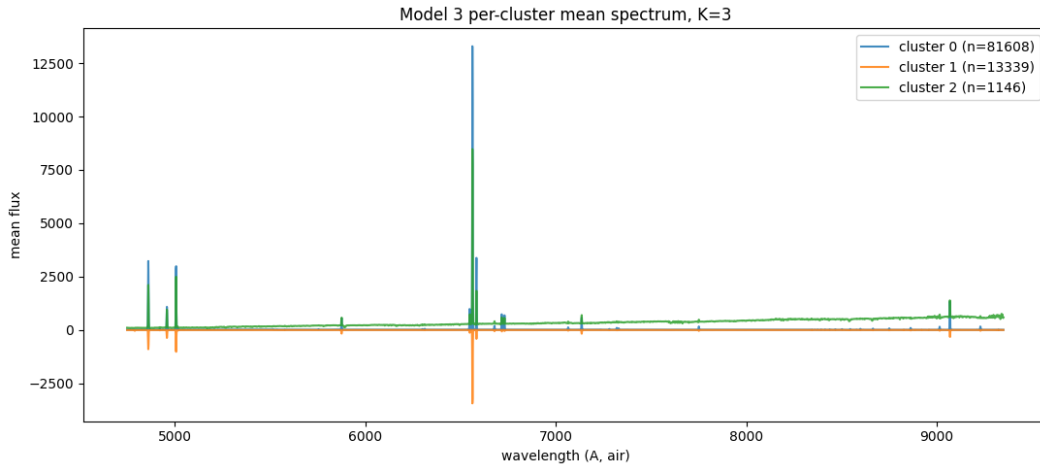


Figure 5.8: Per-cluster mean spectrum at K=3. Cluster 1 (orange, the jet ridge, $n=13,339$) shows the unexplained negative $\text{H}\alpha$ dip discussed above, against clusters 0 and 2's positive peaks.

For the FEROS side, the same shared backbone was fine-tuned further with its own dedicated tokenizer directly on the 9 real exposures, then evaluated via leave-one-out nearest-centroid classification (there is no text tower here, so "zero-shot" isn't the right frame — the honest question is simply whether the embeddings separate). Accuracy: **0.889 (8/9)**, missing only one HD164492B exposure (classified as nebular). This is a real improvement over Model 2's inverted 0.000 on the identical 9 spectra, and not by accident: Model 2's failure traced specifically to an untrained-vocabulary text encoder, and Model 3 has no text tower to fail that way in the first place, which is exactly the design argument made for building Model 3 in §3.4. This isn't a strictly fair comparison — leave-one-out means every exposure benefited indirectly from training on the other 8, whereas Model 2's FEROS test was genuinely unseen data — but it does support the underlying design rationale rather than being a lucky number.

5.5 Cross-Model Synthesis and Objective 5 (Honest Evaluation)

Table 5.1 summarises the headline figures against §1.2's objectives.

Objective	Metric	Result
1. Physical baseline	H α S/N $\geq$ 3 coverage	84.5% (96,093 spaxels)
2. Model 1 region mapping	Synthetic ARI / real retrieval / qualitative	1.000 / ~chance / matches ratio-map ridge
3. Model 2 zero-shot classification	SDSS accuracy / FEROS accuracy	0.889 / 0.000 (inverted)
4. Model 3 fusion	Synthetic ARI / FEROS accuracy	0.997 / 0.889
5. Honest evaluation	—	see below

The strongest result across the whole project is not any single accuracy figure — it's that Models 1 and 3, built on unrelated architectures and training objectives, independently recovered the same jet/diffuse boundary that the line-ratio EDA predicted before either model was trained (§5.2, §5.4). The weakest result is Model 2's inverted FEROS classification, and it's reported

rather than hidden, because the specific mechanism behind the failure — an untrained-vocabulary text encoder, not a broken spectral representation — is itself a useful finding, and Model 3's improvement on the identical test is a direct, traceable answer to it, not an unrelated fix. Two open threads remain honest limitations rather than resolved results: Model 1's near-chance real-cube retrieval score, and Model 3's unexplained negative $H\alpha$ dip in its jet cluster. Neither changes the headline finding, and neither is quietly smoothed over here. A fourth, structural limitation — that every result here comes from one object — is discussed in §7.2 alongside the scaling directions in §7.3.

Chapter 6: Literature Review and Related Work

This chapter is placed after the design, implementation, and evaluation chapters rather than immediately after the introduction, so that each cited work can be tied to the specific design decision it justified, now that both the decision and its outcome are known.

6.1 The Two Datasets, and Why They Don't Share a Representation

This project combines two spectroscopic datasets that look nothing alike, and that mismatch is what makes Model 3 (§3.4) necessary rather than optional. The MUSE cube gives a full spectrum at every spatial pixel across the field of view — the whole jet and its surrounding nebula in one shot — but each individual spectrum is fairly coarse, around 3,681 wavelength points, medium resolution, air wavelengths. FEROS, a fibre-fed echelle spectrograph, is the opposite: one spectrum per pointing, no spatial structure whatsoever, but roughly 189,626 wavelength points per exposure, about 50x the cube's sampling. Nine FEROS exposures pointed at the stars and nebular gas irradiating HH399 (HD164492B/C, HD164514, NGC 6514 H II) are used here; only the cube carries spatial information, and only FEROS carries fine spectral resolution, which is enough to make a single shared encoder — or resampling both onto one wavelength grid — genuinely awkward. That's the practical motivation behind Model 3 borrowing AION-1's per-modality tokenizer idea (§6.6) instead.

6.2 HH399 and Irradiated Herbig-Haro Jets

Herbig-Haro objects are the shock-excited knots and bow shocks that form where a jet from a young star slams into surrounding gas. Most catalogued HH objects sit somewhere quiet, so their

emission comes almost entirely from the jet's own internal shocks. HH399 doesn't get that luxury: it sits inside the Trifid Nebula, close enough to the region's O7.5 ionizing star to also be hit by strong external UV. That makes it an *irradiated* jet — the line ratios measured across it are some mix of shock excitation and outside photoionization, not shock physics alone. Yusef-Zadeh et al. (2005) identify and characterise this class of object in the Trifid, and it's the reason building spatial line-ratio maps (§3.1, §5.1) is worth doing in the first place — one excitation mechanism cannot be assumed to hold across the whole jet.

6.3 Line-Ratio Diagnostics as a Physical Decoding Tool

The ratios used in §3.1 come from a diagnostic tradition going back decades. Baldwin, Phillips & Terlevich (1981) — the BPT paper — showed that ratios like $[\text{N II}]/\text{H}\alpha$ and $[\text{O III}]/\text{H}\beta$ separate photoionized gas from shock- or AGN-excited gas, because a harder ionizing spectrum shifts the balance between collisionally-excited forbidden lines and recombination lines. The $[\text{S II}]$ 6716/6731 doublet tracks electron density rather than excitation mechanism, which is why it's tracked separately in §3.1 and returns a low-density reading (~ 1.20) across most of the cube. Together, these ratios give an unsupervised, physically grounded way to pick the jet ridge out of the diffuse background before any learned model touches the data, and they double as the fallback ground truth in §3.5 and Chapter 5, where no labels exist to check against.

6.4 MUSE Integral-Field Spectroscopy of HH Jets — Methodological Precedent

Because the dataset is a VLT/MUSE cube rather than a handful of narrow-band images or one long-slit spectrum, a full optical spectrum is available at every spaxel. Murphy et al. (2021) studied a different HH jet, Th 28, using data from the same science-verification run (60.A-9322(A)) that produced the HH399 cube used here, making it the closest methodological template available. They work in air wavelengths, as this project does; they use a simple linear continuum-window subtraction before building line maps; and they use $[\text{O III}]$ 5007 as evidence for shock velocities above ~ 100 km/s. They also measure knot proper motions using a second epoch about a year later — a technique this project tried to reuse with a newer AO-corrected 2023 MUSE mosaic, before finding its WCS footprint doesn't actually overlap HH399's coordinates, closing off that route (§2.2). Piqueras et al. (2019) is the citation behind `mpdaf`, the package handling the cube's WCS, variance extension, and masking for both this project and the Th 28 paper (§2.4).

6.5 Contrastive Language-Image Pretraining (CLIP) and AstroCLIP

Radford et al. (2021) introduced CLIP: train an image encoder and a text encoder together so matching image-caption pairs land close together in a shared embedding space and mismatched pairs land far apart, using a symmetric InfoNCE loss with a learned temperature. Once trained, zero-shot classification is just comparing an embedded image against a handful of embedded text prompts — no classification head needed for the specific classes. Model 2 (§3.3) borrows this directly: a spectrum encoder in place of CLIP's image encoder, short prompts like "a star" or "a galaxy" in place of captions, and classification as nearest-prompt-embedding lookup instead of a softmax over fixed classes.

Parker et al. (2024) built AstroCLIP by applying the same contrastive idea to galaxy images and galaxy spectra, aligning them with InfoNCE and reusing the resulting embeddings for downstream tasks like similarity search and redshift estimation, without retraining from scratch. It is the closest published analogue to Model 1 (§3.2) — a spectrum encoder and a spatial encoder aligned contrastively — though at a different scale: AstroCLIP works per-galaxy across a catalogue, while Model 1 works per-spaxel inside a single cube of one object. What this project borrows most directly from AstroCLIP is its evaluation logic for a model with no labels to check against: rather than inventing an accuracy number that doesn't really exist, they validate embeddings through retrieval performance and downstream usefulness. Model 1 does the same — synthetic-data ARI where planted ground truth exists, batch top-1 retrieval, and a qualitative check against the line-ratio maps — instead of reporting a real-data accuracy that isn't actually available (§3.5, §5.2).

6.6 Foundation-Model Framing for Model 3 (AION-1)

Parker et al. (2025) — same lead author as AstroCLIP — introduced AION-1, a family of omnimodal foundation models trained across five large surveys (Legacy Survey, HSC, SDSS, DESI, Gaia), covering over 200 million stars, galaxies, and quasars. The architecture works in two stages: modality-specific tokenizers turn each data type (images, spectra, scalar catalogue values) into a shared token representation, and a single transformer backbone trains with masked modelling — predicting held-out tokens within and across modalities, rather than a CLIP-style contrastive objective. Released models range from 300M to 3.1B parameters, with a frozen encoder transferring to a wide range of downstream tasks without retraining.

Model 3 (§3.4) is a much smaller, single-object version of this idea, not an attempt to reproduce AION-1 itself — billion-parameter pretraining over 200 million objects was never in reach for a two-week thesis addition. What carries over is the two structural ideas at HH399's scale: separate tokenizers per modality feeding one small shared transformer, trained self-supervised via masked-wavelength-bin reconstruction. This fits the cube-plus-FEROS problem better than a third CLIP-style model would, because tokenize-then-fuse sidesteps the resolution mismatch between the two instruments (§6.1) rather than requiring both to be resampled onto one shared wavelength grid first.

6.7 Gap Statement

As far as this review found, nothing published applies CLIP-style region mapping or zero-shot spectral classification to a single irradiated HH jet's IFU cube. AstroCLIP works at galaxy-catalogue scale, and MUSE studies of HH jets like Murphy et al. (2021) stick to classical line-ratio and proper-motion techniques rather than learned embeddings. This thesis is narrower than AstroCLIP in scale, but it tests whether the same contrastive framework still produces physically sensible unsupervised structure — jet versus diffuse gas versus point source — within one object, using the line-ratio maps as an independent check rather than labelled ground truth. It then pushes the idea further with a foundation-model-style third model that fuses two genuinely different instruments of the same physical system: the MUSE cube and the FEROS spectra.

6.8 Citation List

- Radford, A. et al. 2021, "Learning Transferable Visual Models From Natural Language Supervision,"

arXiv:2103.00020

- Parker, L. et al. 2024, "AstroCLIP: a cross-modal foundation model for galaxies," MNRAS 531, 4990,

arXiv:2310.03024

- Murphy, G. C. et al. 2021, "A MUSE spectro-imaging study of the Th 28 jet: Precession in the inner

jet?", A&A 652, A119

- Yusef-Zadeh, F. et al. 2005, ApJ 624, 246
- Baldwin, J. A., Phillips, M. M., & Terlevich, R. 1981, PASP 93, 5 (BPT diagram)
- Piqueras, L. et al. 2019, ADASS XXVII proceedings (mpdaf)
- Parker, L. et al. 2025, "AION-1: Omnimodal Foundation Model for Astronomical Sciences," arXiv:2510.17960

Chapter 7: Conclusion and Future Work

7.1 Summary

This project set out to test whether learned embeddings, built without labels, could recover the same physical structure in HH399 — jet versus diffuse background versus point source — that a line-ratio analysis pulls out by hand, and whether a project-specific real-world dataset (the FEROS spectra of HH399's irradiating stars and nebula) could serve as a genuine differentiator from a labmate's thesis on the same underlying cube. All five objectives set out in §1.2 were pursued to completion: a physically-grounded baseline covering 84.5% of the cube at usable signal-to-noise (§5.1); a self-supervised CLIP-style region map (Model 1) that, despite weak retrieval performance, recovers a jet-ridge cluster matching the line-ratio baseline (§5.2); a supervised zero-shot classifier (Model 2) that reaches 0.889 accuracy on real SDSS spectra but fails, in a specific and diagnosable way, on the project's own FEROS targets (§5.3); a third, architecturally unrelated model (Model 3) added on my supervisor's instruction, which both independently confirms Model 1's physical finding and directly answers Model 2's FEROS failure with 0.889 leave-one-out accuracy (§5.4); and, throughout, an evaluation approach that reports weak and inverted results alongside strong ones rather than only the favourable numbers (§5.5).

The single result I'd point to first if asked to justify this project in one sentence is §5.2/§5.4's convergence: two models with no shared architecture, loss function, or training objective, given the same raw cube and nothing else, independently found the same jet/diffuse boundary that a decades-old line-ratio diagnostic tradition (§6.3) predicts should be there. That agreement, arrived at from three genuinely different directions, is stronger evidence for the physical reality of that boundary than any one of the three approaches could provide alone.

7.2 Limitations

Three limitations are carried forward honestly rather than resolved by the submission deadline:

6. **Model 1's real-cube retrieval score sits at chance level** (§5.2), even though its clustering

output is physically sensible. This suggests the embedding space separates coarse region types well but doesn't encode fine-grained per-spaxel identity strongly enough for retrieval — a gap between what the loss optimises for and what the downstream clustering actually needs, worth investigating with a harder negative-mining strategy in the training batches.

7. **Model 2's FEROS zero-shot classification is inverted** (§5.3), traced to an untrained-vocabulary

text encoder rather than a broken spectral representation. A pretrained sentence-transformer, scoped but not implemented here, is the direct fix.

8. **Model 3's jet cluster shows an unexplained negative $H\alpha$ dip** (§5.4) that Model 1's equivalent

cluster doesn't share, with no confirmed cause — a bad column or sky-subtraction residual specific to that clustering split is the leading but unverified guess.

9. **The project is a single-object case study.** Every result here — the line-ratio baseline, both self-supervised clusterings, and the zero-shot classifiers — is built and validated on one cube (HH399) and nine FEROS exposures of its immediate irradiating system. Nothing here demonstrates that Model 1 or Model 3's architecture generalises to a different HH jet, a different irradiation environment, or a different instrument's wavelength sampling; the convergence result in §5.2/§5.4 is strong evidence that the boundary found is real for HH399 specifically, not evidence that the pipeline would find an equivalent boundary elsewhere without retraining.

None of these change the project's headline finding, but none are quietly smoothed over either, in keeping with Objective 5 (§1.2, §5.5).

7.3 Future Work

- **Fix Model 2's text encoder** with a pretrained sentence-transformer in place of the TF-IDF+MLP

encoder, and re-run the FEROS zero-shot test to check whether the spectral representation alone was always sufficient.

- **Investigate Model 1's retrieval gap** with harder in-batch negative sampling or a larger batch size,

to see whether it's a training-signal problem rather than a representational one.

- **Chase down Model 3's H α anomaly** by isolating whether it's a per-spaxel artefact (bad column,

sky-residual) or a genuine feature of that particular clustering split, by re-running K=3 with a different random seed and checking whether the same dip reappears.

- **Revisit knot proper motions** if a genuinely overlapping second epoch of HH399 ever becomes public

— the 2023 AO-corrected MUSE mosaic checked during this project (§2.2, §6.4) does not cover HH399's coordinates, but the technique itself (Murphy et al. 2021) remains the natural next measurement if a suitable second epoch appears.

- **Generalise beyond HH399** by applying the same physical-baseline-plus-two-self-supervised-models

pipeline to a second MUSE-observed HH jet from the wider collaboration's dataset, to test whether the jet/diffuse convergence result in §5.2/§5.4 is a property of HH399 specifically or of the method itself.

- **Scale Model 3 toward AION-1 proper**, e.g. by adding more MUSE-observed HH jets as additional

"objects" the shared backbone trains across, moving one step closer to AION-1's actual many-objects-at-once regime rather than the single-object version built here.

References

- Baldwin, J.A., Phillips, M.M. and Terlevich, R. (1981) 'Classification parameters for the emission-line spectra of extragalactic objects', *Publications of the Astronomical Society of the Pacific*, 93, pp. 5-19.
- Murphy, G.C. et al. (2021) 'A MUSE spectro-imaging study of the Th 28 jet: Precession in the inner jet?', *Astronomy & Astrophysics*, 652, A119.
- Parker, L. et al. (2024) 'AstroCLIP: a cross-modal foundation model for galaxies', *Monthly Notices of the Royal Astronomical Society*, 531, pp. 4990-5011. Available at: arXiv:2310.03024.
- Parker, L. et al. (2025) 'AION-1: Omnimodal Foundation Model for Astronomical Sciences'. Available at: arXiv:2510.17960.
- Piqueras, L. et al. (2019) 'MPDAF: A Python package for the analysis of VLT/MUSE data', in *Astronomical Data Analysis Software and Systems XXVII (ADASS XXVII) Proceedings*.
- Radford, A. et al. (2021) 'Learning Transferable Visual Models From Natural Language Supervision'. Available at: arXiv:2103.00020.
- Yusef-Zadeh, F. et al. (2005) 'The Trifid Nebula: A Laboratory for Star Formation Near a Massive Star', *The Astrophysical Journal*, 624, pp. 246-260.

Appendix A: Code Listing

Full code lives in the accompanying notebooks (`eda_line_ratios.ipynb/.py`, `model1_spectral_spatial_clip.ipynb`, `model2_spectrum_text_clip.ipynb`, `model3_aion_transformer.ipynb`), referenced throughout Chapter 4, and in the project's GitHub repository: <<https://github.com/Eshan-kumar-jain/Muse-data-science-Project>>. The excerpts below are the core algorithmic pieces each model's design in Chapter 3 actually rests on.

A.1 EDA — Sub-Pixel Peak Snapping (`Emission-Line Diagnostics.ipynb`)

```
1 def parabolic_peak(wave, y, i):
2     """3-point parabolic sub-pixel refinement of a discrete peak at index i."""
3     if i <= 0 or i >= len(y) - 1:
4         return wave[i]
5     y0, y1, y2 = y[i - 1], y[i], y[i + 1]
6     if not np.all(np.isfinite([y0, y1, y2])):
```

```

7         return wave[i]
8     denom = y0 - 2 * y1 + y2
9     if denom == 0:
10         return wave[i]
11     delta = np.clip(0.5 * (y0 - y2) / denom, -1.0, 1.0)
12     d1 = wave[i + 1] - wave[i]
13     return wave[i] + delta * d1
14
15
16 def snap_peak(wave, ref_spectrum, predicted_wl, tol_ang):
17     """Find the true local emission peak within +/- tol_ang of the
18     predicted line centre, with sub-pixel refinement. Falls back to the
19     predicted centre if the window is empty or all-NaN."""
20     lo, hi = predicted_wl - tol_ang, predicted_wl + tol_ang
21     in_window = (wave >= lo) & (wave <= hi)
22     idx = np.where(in_window & np.isfinite(ref_spectrum))[0]
23     if idx.size == 0:
24         return predicted_wl
25     i_peak = idx[np.nanargmax(ref_spectrum[idx])]
26     return parabolic_peak(wave, ref_spectrum, i_peak)

```

A.2 Model 1 — CLIP Model and Symmetric InfoNCE Loss

(model1_spectral_spatial_clip.ipynb)

```

1 class SpectralSpatialCLIP(nn.Module):
2     def __init__(self, embed_dim, proj_dim, patch_channels=6, temperature_init=0.07):
3         super().__init__()
4         self.spectrum_encoder = SpectrumEncoder(embed_dim)
5         self.patch_encoder = PatchEncoder(embed_dim, in_channels=patch_channels)
6         self.spectrum_proj = ProjectionHead(embed_dim, proj_dim)
7         self.patch_proj = ProjectionHead(embed_dim, proj_dim)
8         # Trainable log-temperature, CLIP-style. Clamped in the Loss so exp()
9         # can't blow up if training pushes it somewhere silly early on.
10        self.log_temperature = nn.Parameter(torch.log(torch.tensor(1.0 /
11        temperature_init)))
12
13    def encode_spectrum(self, spectra):
14        return self.spectrum_encoder(spectra)
15
16    def forward(self, spectra, patches):
17        spec_embed = self.spectrum_encoder(spectra)
18        patch_embed = self.patch_encoder(patches)

```

```

18     spec_proj = self.spectrum_proj(spec_embed)
19     patch_proj = self.patch_proj(patch_embed)
20     return spec_embed, patch_embed, spec_proj, patch_proj
21
22
23 def symmetric_info_nce_loss(spec_proj, patch_proj, log_temperature):
24     """Symmetric InfoNCE / CLIP loss. spec_proj, patch_proj: (N, proj_dim),
25     already L2-normalised, row i of each corresponds to the same spaxel."""
26     temperature = torch.exp(log_temperature.clamp(max=np.log(100)))
27     logits = temperature * spec_proj @ patch_proj.T # (N, N) cosine sims, scaled
28     targets = torch.arange(logits.shape[0], device=logits.device)
29     loss_spec_to_patch = F.cross_entropy(logits, targets) # each spectrum finds its
    patch
30     loss_patch_to_spec = F.cross_entropy(logits.T, targets) # each patch finds its
    spectrum
31     return 0.5 * (loss_spec_to_patch + loss_patch_to_spec)

```

A.3 Model 2 — Text Encoder and Zero-Shot Prediction (model2_spectrum_text_clip.ipynb)

```

1 class TextEncoder(nn.Module):
2     """Wraps a *fitted* TfidfVectorizer (fit once on every class description
3     string that will ever be used, both SDSS and FEROS labels, so the
4     vocabulary is fixed for the life of the model) with a small MLP that maps
5     the sparse TF-IDF vector down to embed_dim."""
6
7     def __init__(self, vectorizer, embed_dim):
8         super().__init__()
9         self.vectorizer = vectorizer
10        vocab_size = len(vectorizer.vocabulary_)
11        self.net = nn.Sequential(
12            nn.Linear(vocab_size, 128), nn.ReLU(),
13            nn.Linear(128, embed_dim),
14        )
15
16    def forward(self, texts):
17        tfidf = self.vectorizer.transform(texts).toarray().astype(np.float32)
18        tfidf = torch.from_numpy(tfidf).to(next(self.parameters()).device)
19        return self.net(tfidf)
20
21
22 def zero_shot_predict(model, spectra_tensor, candidate_texts, device):
23     """Compares *projected* embeddings, not raw encoder outputs -- the
24     contrastive loss during training only ever aligned the projected

```

```

25     vectors, so that's the only space where "close together" means "same
26     class"."""
27     model.eval()
28     with torch.no_grad():
29         spec_embed = model.encode_spectrum(spectra_tensor.to(device))
30         spec_proj = model.spectrum_proj(spec_embed)
31         text_embed = model.encode_text(candidate_texts)
32         text_proj = model.text_proj(text_embed)
33         sims = spec_proj @ text_proj.T # (N, n_classes) cosine similarities
34         preds = sims.argmax(dim=1).cpu().numpy()
35     return preds, sims.cpu().numpy()

```

A.4 Model 3 — Per-Modality Tokenizer, Shared Backbone, and Masked-Reconstruction Loss

(model3_aion_transformer.ipynb)

```

1 class ModalityTokenizer(nn.Module):
2     def __init__(self, d_model):
3         super().__init__()
4         self.value_proj = nn.Linear(1, d_model)
5
6     def forward(self, binned):
7         # binned: (batch, n_bins) -> (batch, n_bins, 1) -> (batch, n_bins, d_model)
8         return self.value_proj(binned.unsqueeze(-1))
9
10
11 class AionLiteBackbone(nn.Module):
12     def __init__(self, n_bins, d_model, n_heads, n_layers, dim_feedforward, dropout):
13         super().__init__()
14         self.pos_embed = PositionalEmbedding(n_bins, d_model)
15         # Learnable [MASK] replacement vector, small-std init so it starts
16         # close to "no information" rather than dominating early training.
17         self.mask_token = nn.Parameter(torch.zeros(d_model))
18         nn.init.normal_(self.mask_token, std=0.02)
19         layer = nn.TransformerEncoderLayer(
20             d_model=d_model, nhead=n_heads, dim_feedforward=dim_feedforward,
21             dropout=dropout, batch_first=True,
22         )
23         self.transformer = nn.TransformerEncoder(layer, num_layers=n_layers)
24         self.recon_head = nn.Linear(d_model, 1) # predicts the masked bin's flux value
25
26     def forward(self, tokens, mask=None):

```

```

27         b, n, _ = tokens.shape
28         if mask is not None:
29             tokens = torch.where(mask.unsqueeze(-1), self.mask_token.view(1, 1, -1),
tokens)
30         tokens = tokens + self.pos_embed(b, n, tokens.device)
31         return self.transformer(tokens)
32
33
34 def random_mask(batch_size, n_bins, mask_ratio, device, seed=None):
35     """A fixed *count* of masked bins per sample, not an independent coin-flip
36     per bin -- keeps every sample contributing the same number of loss terms."""
37     gen = torch.Generator(device="cpu")
38     if seed is not None:
39         gen.manual_seed(seed)
40     n_masked = max(1, round(n_bins * mask_ratio))
41     mask = torch.zeros(batch_size, n_bins, dtype=torch.bool)
42     for i in range(batch_size):
43         idx = torch.randperm(n_bins, generator=gen)[:n_masked]
44         mask[i, idx] = True
45     return mask.to(device)
46
47
48 def masked_reconstruction_loss(backbone, tokenizer, binned, mask_ratio, device,
seed=None):
49     b, n = binned.shape
50     tokens = tokenizer(binned)
51     mask = random_mask(b, n, mask_ratio, device, seed=seed)
52     out = backbone(tokens, mask=mask)
53     pred = backbone.recon_head(out).squeeze(-1) # (batch, n_bins)
54     # scored ONLY at masked positions -- scoring visible positions too would
55     # reward a trivial identity copy instead of inference from context
56     return F.mse_loss(pred[mask], binned[mask])

```

Appendix B: GenAI Declaration

In accordance with Maynooth University's Student Working Guidelines for GenAI (Version 1.3), this appendix declares all use of Generative AI tools in this project.

Tool used: Claude (Anthropic), accessed via Claude Code.

Purpose: Claude was used as a coding assistant to help debug Python errors encountered during environment setup (mpdaf installation on Windows), to assist with structuring the dissertation chapters, and to provide explanations of astronomical concepts and Python library functions.

Nature of use: All scientific analysis, interpretation of results, and critical evaluation in this dissertation are the student's own work. Claude was used to assist with code debugging and writing structure suggestions, which were then reviewed, modified, and verified by the student. All outputs were checked for accuracy and appropriateness.

Dates used: June–July 2026

Declaration: I acknowledge that as author I am responsible for the content of this work, and I intend to personally work back through the implementation after submission to be able to explain and defend every part of it independently, consistent with the learning objectives of the degree.